

supervised vs unsupervised algos

supervised->

//Regression

- Linear Regression -> $y=mx+c$ //done
- Logistic Regression -> $z(y=mx+c)$ -> $[0,1]$ -uses sigmoid act fun //done

lass Regression->l1 regularization //done

l2-ridge //done

//classification

- Support Vector Machines (SVMs) ->kernel , hyperplane, support vectors
- Logistic Regression -> $z(y=mx+c)$ -> $[0,1]$ -uses sigmoid act fun //done
- Decision Trees -entropy, information gain, gini impurity (para->depth , criteria)

Random Forests(bagging with node level feature sampling, GridSearchCV, depth, feature, gini importance)->used for feature selection

- Neural networks
- KNN ->k is a parameter, using euclidean distance

Gradient Boosting

Naïve Bayes

$p(a/b) \rightarrow p(a \cap b)/p(b)$

->for independent event -> $p(a \cap b) = p(a) * p(b)$

->for mutual exclusive event -> $p(a \cap b) = 0$

->Bayes theorem -> $p(a/b) = p(b/a) * p(a)/p(b)$

unsupervised->

Clustering

- k-Means ->clusters(known using elbow method) forming using euclidean distance
- Hierarchical Cluster Analysis (HCA)
- Principal Component Analysis (PCA)
- t-distributed Stochastic Neighbor Embedding (t-SNE)

DBSCAN

- Association rule learning

- Apriori

- Eclat

//bias ->diff betn avg pred and correct val

//variance ->change in target fun w.r.t to data

//overfit ->low bias , high var, accuracy high on training data but not on validation data

//underfit ->high bias ,low var

//evaluation methods

//learning rate

//epoch

//hyperparameters

//model parameters

//pandas

//numpy

//matplotlib

//data standardisation

//sklearn

//loss function ->mse squared-> $f(x)=1/n(y_{act}-y_{pred})^2$ ->mainly for Regression eg.linear Regression

->cross entropy-> $f(x)=1/n(y_{act}*\log(y_{pred})+1-y_{act}(\log(1-y_{pred})))$ ->for classification
eg.Logistic Regression

->root(mse)->Regression

->hinge loss->classification eg.svm

//activation function

step function->0 or 1

sigmoid ->[0,1]

tanh->[-1,1]

relu -> $\max(z, 0)$ -> cause dead neuron problem

leaky relu -> $\max(0.01z, z)$

softmax -> multiclassification problem

//weight initialisation techniques

//do not initialise weight to zero -> cause vanishing gradient problem

//const val for weight causes model to not learn non linear data

//very low val causes vanishing gradient problem

//very high val causes unstable weight

techniques -> glort(tanh, sigmoid) -> $\sqrt{1/\text{no of INPUT}}$ and he(relu) -> $\sqrt{2/\text{no of INPUT}}$

//basic structr of ml model

```
class{  
    init();  
    fit();  
    predict();  
    eval();  
}
```

//basic MLP model

```
class{  
    model=model()  
    model.add(layer_type(no_of_nodes, activation function, inputdim)) -> only for first layer  
    model.compile(loss, optimizer, metrics)  
    model.fit(x, y)  
}
```

//regularization ->used to reduce model overfit

l1->lasso -> $(\lambda/n) * (w_i)$ add this term in loss/cost function

l2-ridge-> $(\lambda/n) * (w_i)^2$ add this term in loss/cost function

//batch normalisation->used to make training faster

->normalize output of every hidden layers neuron

$z_{batchnormal} = \lambda_{batchnormal} * z_{normal} + \beta$

here mean and standard deviation are calculated using weighted moving average

//optimizer in deep learning

//vanilla gradient descent is not good because it can give sub optimal solution due to local minima problem

//1.sgd with momentum->tackles local minima problem ->use weighted moving avg(velocity)

$w_{new} = w_{old} - v_t$

$v_t = \beta * v_{t-1} + \text{learning rate} * dw/dt$

//beta generally is 0.9

//some times too much oscillations lead to slower training

//2.nesterov accelerated gradient

//same as sgd with momentum-but we first gain momentum- and then calculate the gradient

//fix the problem- multiple oscillations

//weighted moving avg at time t is

$v_t = \beta * v_{t-1} + (1-\beta) * \theta$

//3.adagrad

//learning rate changes

$w_{new} = w_{old} - \text{learning rate} * dw/dt / \sqrt{v_t + \epsilon}$

//4.adam optimizer

//combination of adagrad and sgd

//use keras tuner for hyperparameters optimization

//vanishing gradient problem->use relu activation function, reduce no of layers

//occurs when dl/dw is very low

//use dropout layers in NN to avoid overfitting(similar to that of ensemble learning)

//new weights will be $w_{new} = w_{old}(1-p)$

//for overfitting in nn , use early stopping methods

//cnn->convolution Neural network

// ->made of convolution layers, pooling layers

// ->here convolution layers extract edges(features) and increasingly find more complex features, until the image is build

>uses convolution unlike matrix multiplication i.e filter which performs the operation

->each filter(3×3 matrix) captures specific features map eg. horizontal filter, vertical filter using relu

->features map= $(n-f+1) \times (n-f+1)$

//filter are training parameters

//some image features gets reduced due to convolution operation

// therefore we perform padding i.e $n=n-f+1$ ->we add extra col in the image

//general formula for feature map= $n+2p-f/s + 1$

//stride->filter jump->default value is 1-p

//high stride-used for high level feature

//padding type->valid and same

//feature map is always single channel

//pooling is used to fix translation variance

//pooling(size, stride, type)->default(2*2,2,max)

//transfer learning

->we freeze the convolution layers and only train fc layers on the new data(feature extraction)(if prev model has similarity with new data)

->we train some last convolution and fc layers(fine tuning)

//functional api

->used to train non linear model

//rnn

->here every node passes its output to the other nodes in its layers and itself

->default activation ->tanh

->text needs to be encoded into integers using ohf or Embedding

->input should be normalized to similar size using padding

//SimpleRNN(32, input_shape=(10, 1)), # 10 timesteps, 1 feature ->weights are updated after batch

//types of rnn

->many to one (input is sequence ,output is scalar eg.sentiment Analysis)

->many to many(seq2seq->same and variable length are its type) (input is sequence ,output is sequence eg.machine translation)

->one to many (input is non sequence ,output is sequence eg.image captioning)

>one to one

->RNN FACES vanishing gradient problem

->it is not good for long text

//seq2seq models

>type of rnn

->async rnn

//Embedding layers

->requires no of unique words as input

->output dim->dim of Vector representing a single word

->input_length->length of input (maxlen)

//deep learning

//mlp

//cnn->image , video Analysis

//rnn->nlp applications

//auto encoders

//gan->

//lstm->maintains two lines ,i.e short(hidden) and long(cell) term path

//advanced to rnn

//has gates ->FORGET, INPUT,OUTPUT

FORGET->BASED ON CURRENT INPUT DECIDES WHAT TO REMOVE FROM THE LONG TERM MEMORY

INPUT->BASED on current input, decides whether to add it in long term MEMORY

OUTPUT->based on current input and what's present in long term MEMORY decides the output

//LSTM(units, return_sequences=False, input_shape=(timesteps(maxlen, features(vector size))))

//it uses sigmoid activation functions and tanh functions

//here the no of nodes is a hyperparameter

//use multiple layers of rnn/lstm/gru for complex tasks

//seq2seq models

>type of rnn

->async rnn

>bert(encoder only)

>gpt(decoder only)

//GRU

->lighter than lstm

->has reset, update gates

->calculate candidate hidden state

//encoder-decoder network

->consist of lstm in both

->last input(hidden state and cell state) of encoder is send to -decoder network as its initial states

-> not good for long text due to loss of context

->encoder has token of input features

->-decoder has token of output features

->uses teacher forcing for training

>uses special symbols like <start>, <end> for -decoder

//attention mechanism

->based on encoder-decoder

->here every hidden state of encoder is available to the -decoder at every timesteps

>requires high computation

>there is a ann network encoder and decoder which calculate the values of the alpha(Bahdanau Attention) using softmax for output layer

> $c_i = \alpha(h_i)$

>

//transformer

>based on encoder decoder

->does not use lstm

->uses self Attention

->multimodal

->parraell training

>eg.bert, gpt

//self Attention->a functions which transform normal Embedding into smart contextual Embedding
consider there 3 vector Embedding x,y,z

$x_{new} = (x \cdot x^T)x + (x \cdot y^T)y + (x \cdot z^T)z$ // similar for y_{new} and z_{new} , it is also normalized using softmax
,better for generalised task

$x_{new} = (x_{query} \cdot x_{key})x_{value} + (x_{query} \cdot y_{key})y_{value} + (x_{query} \cdot z_{key})z_{value}$ ->new derived for better
specific context vector

//it is done parraell for all the word Embedding

$x_{query} = queryMatrix * x$

$x_{value} = valueMatrix * x$

$x_{key} = keyMatrix * x$

// the entries of this matrix are the training parameters

scaled Attention= $\text{softmax}(\text{queryMatrix} * \text{keyMatrix}^T) * \text{valueMatrix} / \sqrt{d_k}$ -> d_k is dim of word vector Embedding used for reducing variance

//we multi head Attention->multiple self Attention- blocks with linear transformation(training parameters) at the end

//which captures multiple meaning of a sentence

//positional encoding ->words relative position maintains

>we do point wise additn of vector Embedding and positional encoding(using sine and cosine functons using $\sin(\text{pos} / (10000^{** ((2 * i) / d_model)})$ or $\cosine(\text{pos} / (10000^{** ((2 * i) / d_model)})$ before sending it in the self Attention- blocks

//we do not use batch normalized in transformer because present of extra padding to make sentence size equals

//layer normalized ->normalized on features

//transformer encoder

- 1.input with positional encoded vector Embedding
- 2.multi head self Attention
- 3.residual network
- 4.feed forward Neural network
- 5.layer normalized

//transformer-decoder

->its autoregressive for inference and non autoregressive while training by using masked self Attention

->it uses cross Attention to have similarity between input and output sentence,input vector Embedding give key and values , output vector Embedding give query

//transformer encoder

- 1.input with right shifted(adding <start> token) positional encoded vector Embedding
- 2.masked multi head self Attention
- 3.residual network
- 4.cross Attention
- 5.feed forward Neural network
- 6.layer normalized
- 7.output layer(nodes=vocablury of output) with linear and softmax ,output with highest probbality at that timesteps

//transfomer inference

->1.<sos> token given to the decoder

>output of decoder is given as input to decoder at next timesteps

Pandas Important funcntions

- 1.pd.read_csv()
- 2.pd.head()/tail()
- 3.pd.shape
- 4.pd.Value_counts()
- 5.pd.dropna()
- //df.info()
- 6.pd.fillna()
- 7.df.isnull().sum() ->check Missing vlaues in cols
- 8.df.corr()

Data Preprocessin steps

- 1.Handling Missing Values
- 2.Handling Duplicates

3.Noise Removal

4.Normalization / Feature Scaling ->minmax(0,1), standard

5.Feature Extraction/selection

Data Gathering->csv,json.sql,fetch api,webscrapping

//emthod to read csv

df=pd.read_csv("filename" or "filepath",names=[""])->use name para only if give data does not have labels

//study of bagging ,boosting

//pca

creates new feautres and selects them for training as part of feature extraction

reduces dimension of data

eigen vector->direction remain same even after linear transformations

pca components<=no of input feature->geerally we specify its Value

we select the components whose collective sum variance is 90%

//ensemble learning(used for performance imporvement,reduce bais and variance)

//ouput ->for classification->majority

->for regression->mean Value of all the ouput

1.vooting

->diff ml models but same data

->hard and soft vooting

2.bagging(random forest)

->same models but diff data

->can use row as well as column sampling

3.stacking

->diff mododel diff data but extra model which learns from these ouput of these models

4.boosting(adaboost, xgboost, gradient boosting)

adaboost->uses descion steams (weak leaerners)

->upsampling of the data that is incorrectly specified

->transfers learning to the next same models

->based on sign

gradient boosting->each model is weighted

->uses learning rate.

->use concept of residual

xgboost

->based on gradient boosting-

->improvement in speed , performance, accessibility

->can be used on all types of classification/regression-